

Security Finding Report

FalconStrix – Host Intrusion Detection & Response Dashboard

Report ID	FALSTRX-2026-001
Target Application	FalconStrix SOC Dashboard (Flask / Socket.IO)
Affected Component	gui_dashboard/app.py – /api/debug/* endpoints
Vulnerability Class	Broken Access Control – Missing Authentication for Critical Function (CWE-306)
Severity	High
CVSS 3.1 (estimated)	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Discovered By	Manual black-box testing (Burp Suite) + source-code review
Environment	Local isolated lab – Windows 11 target, Kali Linux (VMware) attacker
Date Discovered	22 July 2026
Date Fixed & Verified	22 July 2026
Status	Remediated – Verified

1. Summary

The FalconStrix dashboard exposes a set of internal debug endpoints (`/api/debug/events` and `/api/debug/last-snapshot`) that are explicitly excluded from the application's authentication check. As a result, any unauthenticated client with network access to the application can retrieve live internal telemetry — including system state, active alert counts, process-kill counters, and network throughput statistics — without ever logging in.

2. Vulnerability Details

The application enforces authentication in a global `before_request` handler in `gui_dashboard/app.py`. This handler whitelists a set of routes that should not require a session, intended for genuinely public pages such as the login and signup screens. The debug API prefix was mistakenly included in this same whitelist:

```
if (
    request.path.startswith('/api/debug')
    or request.path.startswith('/api/auth/')
    or request.path.startswith('/socket.io')
    or request.endpoint in ('hero', 'demo', 'login', 'signup',
    'static', 'api_username_availability')
    or request.path.startswith('/static/')
):
```

```
return None
if not session.get('user_id'):
return redirect(url_for('login', next=request.path))
```

Because `/api/debug` is matched before the session check runs, requests to any route under that prefix bypass authentication entirely — regardless of whether a valid session cookie is present.

3. Steps to Reproduce

- Start the FalconStrix application and confirm it is reachable over the network (tested locally at `http://192.168.30.1:5001`).
- Using Burp Suite (or any HTTP client), send a GET request to `/api/debug/events` with **no session cookie present** (i.e. as a fully anonymous, logged-out client).
- Observe the HTTP response.

Request sent (Burp Repeater):

```
GET /api/debug/events HTTP/1.1
Host: 192.168.30.1:5001
User-Agent: Mozilla/5.0
Accept: */*
Connection: close
```

[No Cookie header sent]

Response received:

```
HTTP/1.1 200 OK
Server: Werkzeug/3.1.7 Python/3.11.9
Content-Type: application/json
Content-Length: 21172
```

```
{
  "events": [
    {
      "event_type": "dashboard_snapshot_emit",
      "payload": {
        "active_alerts": 0,
        "ipc_total": 2,
        "kill_cnt": 2,
        "recent_events": 8,
        "state": "NORMAL"
      },
      "ts": "2026-07-22T16:31:26"
    },
    {
      "event_type": "network_stats_emit",
      "payload": {
        "in": "1.68 KB/s",
        "out": "10.70 KB/s",
        "total_in": "1.67 GB",
        "total_out": "2.44 GB"
      },
    },
  ]
}
```

```
"ts": "2026-07-22T16:31:27"
}
... (70+ additional entries returned per request)
],
"ok": true
}
```

The server returned a full 200 OK with detailed internal telemetry despite the absence of any authentication credentials. The identical result was reproduced against `/api/debug/last-snapshot`.

Control test (confirms this is not a general auth failure):

The same no-cookie request technique was repeated against a protected route not covered by the whitelist (`/dashboard`), which correctly returned a 302 Found redirect to `/login`. This confirms the authentication mechanism itself is functioning correctly and that the debug endpoints represent a specific, isolated oversight rather than a systemic authentication failure.

4. Impact

- **Unauthenticated information disclosure:** an external attacker with only network reachability (no credentials) can continuously poll internal SOC telemetry, including live network throughput and cumulative data transferred.
- **Operational reconnaissance:** the exposed state field (e.g. NORMAL vs. an elevated/locked state) reveals whether the monitoring system currently believes it is under attack, which could allow an attacker to time further actions against the host it is protecting.
- **Ironic exposure:** the vulnerable component is itself part of a security monitoring product, meaning the tool intended to detect intrusions leaks its own operational state to unauthenticated parties.
- No direct code execution, credential theft, or data modification was demonstrated as part of this finding; impact is limited to information disclosure and reconnaissance value.

5. Severity Justification

Factor	Rating	Reasoning
Attack Vector	Network	Reachable over the network with no local access required.
Attack Complexity	Low	A single unauthenticated HTTP GET request is sufficient.
Privileges Required	None	No account or session cookie needed.
User Interaction	None	Fully automatable; no victim interaction required.
Confidentiality Impact	High	Full internal telemetry disclosed to anonymous requester.
Integrity / Availability Impact	None	Endpoint is read-only; no state change observed.

Estimated CVSS 3.1 vector: `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N` → Base Score **7.5 (High)**.

6. Remediation

Primary fix – remove debug routes from the public whitelist:

```
if (
request.path.startswith('/api/auth/')
or request.path.startswith('/socket.io')
or request.endpoint in ('hero', 'demo', 'login', 'signup',
'static', 'api_username_availability')
or request.path.startswith('/static/')
):
return None
if not session.get('user_id'):
return redirect(url_for('login', next=request.path))
```

Defense-in-depth – restrict debug data to admin roles and non-production environments:

```
DEBUG_ROUTES_ENABLED = os.environ.get(
'FALCON_ENABLE_DEBUG_ROUTES', 'false'
).lower() == 'true'

@app.route('/api/debug/events')
def api_debug_events():
if not DEBUG_ROUTES_ENABLED:
return jsonify({'ok': False, 'error': 'not found'}), 404
if (session.get('role') or '').lower() != 'admin':
return jsonify({'ok': False, 'error': 'forbidden'}), 403
# ... existing logic ...
```

Apply the same role check to `/api/debug/last-snapshot`. This ensures that even if the whitelist is inadvertently reintroduced in a future change, debug data still cannot be retrieved without both a valid admin session and an explicit environment flag enabling the routes.

7. Verification (Post-Fix Re-Testing)

The remediation applied two independent controls: (1) removal of `/api/debug` from the global `before_request` whitelist, so unauthenticated requests are caught by the existing session check; and (2) an explicit admin-role check added inside each debug route as defense-in-depth. Both controls were verified independently.

Test A – Fully anonymous request (no session cookie):

```
GET /api/debug/events HTTP/1.1
Host: 192.168.30.1:5001
[No Cookie header sent]

--- Response ---
HTTP/1.1 302 FOUND
Location: /login?next=/api/debug/events
```

Result: the request is intercepted by the global authentication check before it ever reaches the route handler, and the client is redirected to the login page. This confirms Control 1 (whitelist removal) is effective on its own against a fully unauthenticated attacker — the exact scenario originally exploited.

Test B – Authenticated as a non-admin user:

```
GET /api/debug/events HTTP/1.1
Host: 192.168.30.1:5001
Cookie: session=<valid session for a non-admin account>
```

```
--- Response ---
HTTP/1.1 403 FORBIDDEN
Content-Type: application/json
```

```
{
  "error": "forbidden",
  "ok": false
}
```

Result: a logged-in but non-privileged account passes the first authentication gate but is correctly blocked by the admin-role check (Control 2), returning 403 Forbidden rather than the internal telemetry. This confirms the second, independent layer of defense functions correctly even for authenticated accounts that lack sufficient privilege, and is distinct from the un-authenticated case (302 vs. 403), matching expected REST semantics: 302/401 for “not logged in” vs. 403 for “logged in but not authorized.”

Both tests were repeated against `/api/debug/last-snapshot` with identical results. Prior to remediation, both endpoints returned 200 OK with full telemetry data under both test conditions; post-remediation, unauthenticated and non-admin requests are consistently rejected.

Before / After Summary

Test Condition	Before Fix	After Fix
No session cookie (anonymous)	200 OK – data disclosed	302 Found – redirect to /login
Valid session, non-admin role	200 OK – data disclosed	403 Forbidden – access denied
Valid session, admin role	200 OK – data disclosed	200 OK – data returned (expected)

Note: the admin-role positive case (row 3) was reasoned through based on code review of the applied fix; it follows directly from the logic verified in the two negative tests above and was not separately screenshotted.

8. Testing Environment

- Target: FalconStrix run locally on a Windows 11 host, bound to `0.0.0.0:5001`, reachable only over an isolated VMware virtual network.
- Attacker: Kali Linux running in VMware, using Burp Suite Community Edition as an intercepting proxy.
- All testing was performed exclusively against a locally-hosted instance of the researcher's own project, with no third-party systems or production deployments involved.